

Solucionario del EXAMEN PARCIAL**CICLO 2021 - I****ARQUITECTURA DE COMPUTADORES (CC221)*****Por: César Martín Cruz Salazar***

1. (4 pts.) Calcule los valores de los registros TH0 y TL0 en Modo 0 y en Modo 1 para un reloj de sistema de 11.0592Mhz y para una frecuencia de 958hz.

Sol.

Se hace el cálculo para generar una frecuencia de 958hz. Se aplica la relación para el semiperiodo 0.00052192066805846 por tanto:

En Modo 0

$$(8192-j) \times (12/11.0592 \times 10^6) \text{s} = 0.00052192066805846 \text{s}$$

$j=7711$ en modo 0 → **TH0=1Eh, TL0=1Fh**

En Modo 1

$$(65536-j) \times (12/11.0592 \times 10^6) \text{s} = 0.00052192066805846 \text{s}$$

$j=65055$ en modo 1 → **TH0=FEh, TL0=1Fh**

2. (4 pts.) Indique al menos 5 diferencias entre el microcontrolador MSP430G2553 y el microcontrolador 8051.

Sol.

8051	MSP430G2553
Un microcontrolador de 8 bits	Un microcontrolador de 16 bits
4kbytes de memoria Flash Rom	16kbytes de memoria Flash Rom
128bytes de memoria de datos	512bytes de memoria de datos
Arquitectura CISC	Arquitectura RISC
No tiene un convertidor A/D	Tiene un convertidor A/D de 10 bits
No tiene un sensor de temperatura	Contiene un sensor de temperatura

3. (4 pts.) Desarrolle un programa que calcule el **octavo** y el **treceavo** término de la serie de Fibonacci(1,1,2,3,5,8,13,21,34,55,89,144,233). Enviar ambos resultados calculados al display LCD. Indicar el UPDATE FREQ a usar.

Sol.

```

; constantes
LCD_DATA equ 090h ; puerto 1 es usado para datos o instrucciones
LCD_DB4 equ 094h ; high nibble del puerto 1(P1.4) es usado para datos o instrucciones
LCD_DB5 equ 095h ; high nibble del puerto 1(P1.5) es usado para datos o instrucciones
LCD_DB6 equ 096h ; high nibble del puerto 1(P1.6) es usado para datos o instrucciones
LCD_DB7 equ 097h ; high nibble del puerto 1(P1.7) es usado para datos o instrucciones
LCD_RS equ 093h ; p1.1 LCD linea Register Select
LCD_RW equ 091h ; p1.2 LCD linea Read / Write
LCD_E equ 092h ; p1.3 LCD linea Enable

```

```

;instrucciones del sistema
Config equ 28h ; 4 bits de datos, 2 lineas, 5 por 7 caracter matriz
entryMode equ 6 ; incrementar cursor, no traslada display, incremento automático del registro AC al escribir.

; instrucciones de control del cursor
offCur equ 0Ch
lineCur equ 0Eh
blinkCur equ 0Dh
combnCur equ 0Fh
homeCur equ 02h ;se coloca el cursor al principio de la línea. Cursor a la izquierda.
shLfCur equ 10h
shRtCur equ 14h

; instrucciones de control del display
clrDsp equ 01h ;Limpia el display(los contenidos son borrados),borra display, cursor al rincón, y pone el modo
;de entrada en incrementar.
offDsp equ 0Ah ;blanquea el display(display se apaga)no hay efecto de parpadeo y el cursor es visualizado
onDsp equ 0Eh ;Muestra mensaje en el display(el display hace aparecer los caracteres)
shLfDsp equ 18h ;Desplazamiento del display a la izquierda
shRtDsp equ 1Ch ;Desplazamiento del display a la derecha
;Update Freq:100

        org 0000h
; inicializa el módulo LCD -----
start:
        lcall inicio_display
        lcall initLCD4 ; inicializa fonts, # de lineas, Modo de entrada
;Mi programa va a partir de aquí:
;octavo y treceavo valor de Fibonacci
        lcall prtLCD4
        db "8avo "
        db 3Ah,0
        mov A,#2
        mov B,#0
        lcall placeCur4
        lcall prtLCD4
        db "13avo "
        db 3Ah,0
        ;a,b y c=a+b
        mov R3,#11
        mov R4,#2
        mov R0,#1      ;R0 hace las veces de a
        mov R1,#1      ;R1 hace las veces de b
lazo:
        mov A,R0
        add A,R1
        mov R2,A      ;R2 hace las veces de c
        inc R4
        cjne R4,#8,sigue
        mov A,#1
        mov B,#6
        lcall placeCur4
        mov A,R2
        lcall dos_digitos_decimales
        mov R0,30h
        lcall wrLCDdata4
        mov R0,31h
        lcall wrLCDdata4
sigue:
        cjne R4,#13,sigue_
        mov A,#2
        mov B,#7
        lcall placeCur4
        mov A,R2
        lcall tres_digitos_decimales
        mov R0,30h
        lcall wrLCDdata4
        mov R0,31h
        lcall wrLCDdata4

```

```

        mov R0,32h
        lcall wrLCDdata4
        mov R0,#offCur
        lcall wrLCDcom4
sigue_:
        mov A,R1
        mov R0,A      ;a=b
        mov A,R2
        mov R1,A      ;b=c
        djnz R3,lazo
        sjmp $

; ::::::::::::::::::::::::::::::::::::::
; subrutinas
; ::::::::::::::::::::::::::::::::::::::
retardo2:
        mov R5,#5
vuelve:
        mov R3,#100
        djnz R3,$
        djnz R5,vuelve
        ret

;=====
;Devuelve tres codigos ASCII de tres digitos decimales
;=====
tres_digitos_decimales:
        mov B,#100
        div AB
        add A,#30h
        mov 30h,A
        mov A,B
        mov B,#10
        div AB
        add A,#30h
        mov 31h,A
        mov A,B
        add A,#30h
        mov 32h,A
        ret

;=====
;Devuelve dos codigos ASCII de dos digitos decimales
;=====
dos_digitos_decimales:
        mov B,#10
        div AB
        add A,#30h
        mov 30h,A
        mov A,B
        add A,#30h
        mov 31h,A
        ret

;=====
;Subrutina binasc
;Toma el contenido del acumulador y
;lo convierte en dos números ascii hexadecimales.
;El resultado es retornado en el acumulador y R2
;=====
binasc:
        mov R2,A ;se mueve a R2 para salvar el valor de A
        anl A,#0Fh ;se convierte el dígito menos significativo
        add A,#0F6h ;lo ajusta
        jnc noadj1
        add A,#07h
noadj1:
        add A,#3Ah ;lo hace ascii
        xch A,R2 ;pone el resultado en R2
        swap A ;se convierte ahora el digito más significativo
        anl A,#0Fh
        add A,#0F6h ;lo ajusta

```

```

        jnc noadj2
        add A,#07h
noadj2:
        add A,#3Ah ;lo hace ascii
        ret

;=====
;Subrutina que se llama al inicio de su programa
;=====
inicio_display:
        CLR P1.3 ; clear RS - indica que instrucciones estan siendo enviados al módulo
        CLR P1.7 ;
        CLR P1.6 ;
        SETB P1.5 ;
        CLR P1.4 ;
        SETB P1.2 ;
        CLR P1.2 ;
        lcall retardo;
        SETB P1.2 ;
        CLR P1.2 ;
        SETB P1.7 ;
        SETB P1.2 ;
        CLR P1.2 ;
        lcall retardo ;
        ret

;=====
;Subrutina de retardo de tiempo
;=====
retardo:
        mov R0,#50
        djnz R0,$
        ret

; =====
; subrutina crShLf4
; Esta rutina traslada el cursor a la izquierda. El numero de
; posiciones a ser trasladado es localizado en el acumulador.
; entrada : acumulador indica numero de posiciones a trasladar
; salida : nada
; =====
;
crShLf4:
        jz ret_scl
        mov r0, #shLfCur ; palabra de control para trasladar a la izquierda
        acall wrLCDcom4
        dec acc
        sjmp crShLf4
ret_scl:
        ret

; =====
; subrutina crShRt4
; Esta rutina traslada el cursor a la derecha. El numero de
; posiciones a ser trasladado es localizado en el acumulador.
; entrada : acumulador indica el numero de posiciones a trasladar
; salida : nada
; =====
;
crShRt4:
        jz ret_scr
        mov r0, #shRtCur ; palabra de control para trasladar a la derecha
        acall wrLCDcom4
        dec acc
        sjmp crShRt4
ret_scr: ret

; =====
; subrutina placeCur4
; Esta rutina fija la posicion del cursor. La posicion del cursor
; es localizado en el registro b. La posicion del cursor empieza
; en 0. El acumulador da el numero de linea.
; entrada: acumulador indica el numero de linea (1, 2)

```

```

; : el registro b guarda la posicion del cursor
; salida : nada
; =====
;
;
placeCur4:
    dec acc ; acc=0 para linea=1
    jnz line2 ; si acc=0 luego primera linea
    mov a, b
    add a, #080h ; palabra de control para la linea 1
    sjmp setcur

line2:
    mov a, b
    add a, #0C0h ; palabra de control para la linea 2

setcur:
    mov r0, a ; localizar palabra de control
    acall wrLCDcom4 ;
    ret

; =====
; subrutina dspShLf4
; Esta rutina traslada los contenidos del LCD a la izquierda. El
; numero de caracteres a ser trasladados es localizado en el
; acumulador.
; entrada : acumulador indica el numero de caracteres a trasladar
; salida : nada
; =====
;
;
dspShLf4:
    jz ret_sdl
    mov r0, #shLfDsp ; palabra de control para trasladar a la izquierda
    acall wrLCDcom4
    dec a
    sjmp dspShLf4

ret_sdl: ret

; =====
; subrutina dspShRt4
; Esta rutina traslada los contenidos del LCD a la derecha. El
; numero de caracteres a ser trasladados es localizado en el
; acumulador.
; entrada : acumulador indica el numero de caracteres a trasladar
; salida : nada
; =====
;
;
dspShRt4:
    jz ret_sdr
    mov r0, #shRtDsp ; palabra de control para trasladar a la derecha
    acall wrLCDcom4
    dec a
    sjmp dspShRt4

ret_sdr: ret

; =====
; subrutina wrLCDdata4
; escribe un dato al LCD
; el dato debe ser localizado en r0 para llamar el programa
; -----
wrLCDdata4:
    clr LCD_E
    setb LCD_RS ; selecciona enviar Dato
    clr LCD_RW ; selecciona operacion de escritura
    push acc ; salva acumulador
    mov a, r0 ; pone byte de datos en el acc
    mov c, acc.4 ; carga high nibble sobre el bus de datos
    mov LCD_DB4, c ; un bit a la vez...
    mov c, acc.5 ;
    mov LCD_DB5, c
    mov c, acc.6
    mov LCD_DB6, c
    mov c, acc.7
    mov LCD_DB7, c

```

```

setb LCD_E ; pone a uno el pin Enable
clr LCD_E ; pone a cero el pin Enable
mov c, acc.0 ; similarmente, carga el low nibble
mov LCD_DB4, c
mov c, acc.1
mov LCD_DB5, c
mov c, acc.2
mov LCD_DB6, c
mov c, acc.3
mov LCD_DB7, c
setb LCD_E ; pone a uno el pin Enable
clr LCD_E ; pone a cero el pin Enable
lcall retardo
pop acc
ret
; =====
; subrutina wrLCDcom4
; escribe una palabra de comando al LCD
; comando debe estar localizado en r0 para llamar el programa
; -----
wrLCDcom4:
clr LCD_E
clr LCD_RS ; selecciona enviar comando
clr LCD_RW ; selecciona operaci n de escritura
push acc ; salva el acumulador
mov a, r0 ; pone el byte de datos en el acc
mov c, acc.4 ; carga el high nibble en el bus de datos
mov LCD_DB4, c ; un bit a la vez usando...
mov c, acc.5 ; operaciones de mover bit
mov LCD_DB5, c
mov c, acc.6
mov LCD_DB6, c
mov c, acc.7
mov LCD_DB7, c
setb LCD_E ; pulsa la l nea Enable
clr LCD_E
mov c, acc.0 ; similarmente, carga el low nibble
mov LCD_DB4, c
mov c, acc.1
mov LCD_DB5, c
mov c, acc.2
mov LCD_DB6, c
mov c, acc.3
mov LCD_DB7, c
setb LCD_E ; pulsa la l nea Enable
clr LCD_E
lcall retardo
pop acc
ret
; =====
; subrutina initLCD4 - inicializa el LCD
;
; -----
initLCD4:
clr LCD_RS ; LCD linea de Register Select
clr LCD_RW ; linea de Read / Write
clr LCD_E ; linea de Enable
mov r0, #entryMode ; fijar Entry Mode
acall wrLCDcom4 ; incrementar cursor a la derecha, no traslada el display
mov r0, #onDsp ; Enciende el display, home cursor
lcall wrLCDcom4
ret
; =====
; subrutina prtLCD4
; Toma la cadena inmediatamente que sigue a la llamada a esta
; subrutina y lo displaya sobre el LCD. La cadena es leida con la
; instrucc n mov a, @a+dptr
; de este modo, la cadena esta en la memoria de datos.
;

```

```

; entrada : nada
; salida : nada
; destruye : acc, dptr
; =====
;
prtLCD4:
pop dph ; regresa direcci3n de retorno en dptr
pop dpl
prtNext:
clr a ; set offset = 0
movc a, @a+dptr ; consigue chr de la memoria de c3digo
cjne a, #0, chrOK ; si chr = 0 entonces retorna
sjmp retPrtLCD
chrOK:
mov r0, a
acall wrLCDdata4 ; enviar caracter
inc dptr ; apuntar al siguiente caracter
ajmp prtNext ; lazo hasta que finalize la cadena
retPrtLCD:
mov a, #1h ; apunta a la instrucci3n despu,s de la cadena
jmp @a+dptr ; retorno con una instrucci3n de salto

end

```

4. (8 pts.) En una máquina de café, se tiene un display LCD de 2x16. Se necesita programar una barra de progreso de la preparación del café y un mensaje cuando se encuentre listo el café.

Línea1 que diga: Preparando....

En la línea 2 se muestra una barra de progreso animado.

Luego, al finalizar la barra de progreso que se muestre:

Línea 1: LISTO!!, línea 2: recoja el café

Indicar el UPDATE FREQ a usar.

Sol.

```

; constantes
LCD_DATA equ 090h ; puerto 1 es usado para datos o instrucciones
LCD_DB4 equ 094h ; high nibble del puerto 1(P1.4) es usado para datos o instrucciones
LCD_DB5 equ 095h ; high nibble del puerto 1(P1.5) es usado para datos o instrucciones
LCD_DB6 equ 096h ; high nibble del puerto 1(P1.6) es usado para datos o instrucciones
LCD_DB7 equ 097h ; high nibble del puerto 1(P1.7) es usado para datos o instrucciones
LCD_RS equ 093h ; p1.1 LCD linea Register Select
LCD_RW equ 091h ; p1.2 LCD linea Read / Write
LCD_E equ 092h ; p1.3 LCD linea Enable

;instrucciones del sistema
Config equ 28h ; 4 bits de datos, 2 lineas, 5 por 7 caracter matriz
entryMode equ 6 ; incrementar cursor, no traslada display, incremento automático del registro AC al escribir.

; instrucciones de control del cursor
offCur equ 0Ch
lineCur equ 0Eh
blinkCur equ 0Dh
combnCur equ 0Fh
homeCur equ 02h ;se coloca el cursor al principio de la línea. Cursor a la izquierda.
shLfCur equ 10h
shRtCur equ 14h

; instrucciones de control del display
clrDsp equ 01h ;Limpia el display(los contenidos son borrados),borra display, cursor al rinc3n, y pone el modo
;de entrada en incrementar.
offDsp equ 0Ah ;blanquea el display(display se apaga)no hay efecto de parpadeo y el cursor es visualizado
onDsp equ 0Eh ;Muestra mensaje en el display(el display hace aparecer los caracteres)
shLfDsp equ 18h ;Desplazamiento del display a la izquierda
shRtDsp equ 1Ch ;Desplazamiento del display a la derecha
;Update Freq:200

org 0000h

```

```

; inicializa el módulo LCD -----
start:
    lcall inicio_display
    lcall initLCD4 ; inicializa fonts, # de lineas, Modo de entrada
;Mi programa va a partir de aquí:
    lcall prtLCD4
    db "Preparando.... "
    db 0
    lcall barra_de_progreso
    mov A,#1
    mov B,#0
    lcall placeCur4
    lcall prtLCD4
    db "      "
    db 0
    mov A,#2
    mov B,#0
    lcall placeCur4
    lcall prtLCD4
    db "      "
    db 0
    mov A,#1
    mov B,#0
    lcall placeCur4
    lcall prtLCD4
    db " LISTO!! "
    db 0
    mov A,#2
    mov B,#0
    lcall placeCur4
    lcall prtLCD4
    db " recoja el cafe "
    db 0
    sjmp $
barra_de_progreso:
    lcall caracter_rect
    mov A,#2
    mov B,#0
    lcall placeCur4
    mov R7,#16
otra_vez:
    lcall prtLCD4
    db 8,0
    lcall retardo2
    djnz R7,otra_vez
    ret
caracter_rect:
    mov A, #0 ; selecciono el caracter 0
    mov B, #0 ; selecciono la fila del tipo 0
    lcall setCGRAM4
    lcall prtLCD4
    db 1fh,1fh,1fh,1fh,1fh,1fh,1fh,20h,0
    ret
; ::::::::::::::::::::::::::::::::::::
; subrutinas
; ::::::::::::::::::::::::::::::::::::
retardo2:
    mov R5,#5
vuelve:
    mov R3,#100
    djnz R3,$
    djnz R5,vuelve
    ret
;=====
;Devuelve tres codigos ASCII de tres digitos decimales
;=====
tres_digitos_decimales:
    mov B,#100
    div AB

```



```

        add A,#30h
        mov 30h,A
        mov A,B
        mov B,#10
        div AB
        add A,#30h
        mov 31h,A
        mov A,B
        add A,#30h
        mov 32h,A
        ret
;=====
;Devuelve dos codigos ASCII de dos digitos decimales
;=====
dos_digitos_decimales:
        mov B,#10
        div AB
        add A,#30h
        mov 30h,A
        mov A,B
        add A,#30h
        mov 31h,A
        ret
setCGRAM4:
        push b
        mov b, #8
        mul ab ; multiplica numero del caracter por 8
        pop b ; b sostiene numero de fila
        add a, b ; en a se sostiene CGRAM address
        add a, #40h ; convert to instruction
        mov r0, a ; place instruction
        acall wrLCDcom4 ; issue command
        ret
;=====
;Subrutina binasc
;Toma el contenido del acumulador y
;lo convierte en dos números ascii hexadecimales.
;El resultado es retornado en el acumulador y R2
;=====
binasc:
        mov R2,A ;se mueve a R2 para salvar el valor de A
        anl A,#0Fh ;se convierte el dígito menos significativo
        add A,#0F6h ;lo ajusta
        jnc noadj1
        add A,#07h
noadj1:
        add A,#3Ah ;lo hace ascii
        xch A,R2 ;pone el resultado en R2
        swap A ;se convierte ahora el digito más significativo
        anl A,#0Fh
        add A,#0F6h ;lo ajusta
        jnc noadj2
        add A,#07h
noadj2:
        add A,#3Ah ;lo hace ascii
        ret
;=====
;Subrutina que se llama al inicio de su programa
;=====
inicio_display:
        CLR P1.3 ; clear RS - indica que instrucciones estan siendo enviados al módulo
        CLR P1.7 ;
        CLR P1.6 ;
        SETB P1.5 ;
        CLR P1.4 ;
        SETB P1.2 ;
        CLR P1.2 ;
        lcall retardo;

```

```

        SETB P1.2 ;
        CLR P1.2 ;
        SETB P1.7 ;
        SETB P1.2 ;
        CLR P1.2 ;
        lcall retardo ;
        ret
;=====
;Subrutina de retardo de tiempo
;=====
retardo:
        mov R0,#50
        djnz R0,$
        ret
; =====
; subrutina crShLf4
; Esta rutina traslada el cursor a la izquierda. El numero de
; posiciones a ser trasladado es localizado en el acumulador.
; entrada : acumulador indica numero de posiciones a trasladar
; salida : nada
; =====
;
crShLf4:
        jz ret_scl
        mov r0, #shLfCur ; palabra de control para trasladar a la izquierda
        acall wrLCDcom4
        dec acc
        sjmp crShLf4
ret_scl:
        ret
; =====
; subrutina crShRt4
; Esta rutina traslada el cursor a la derecha. El numero de
; posiciones a ser trasladado es localizado en el acumulador.
; entrada : acumulador indica el numero de posiciones a trasladar
; salida : nada
; =====
;
crShRt4:
        jz ret_scr
        mov r0, #shRtCur ; palabra de control para trasladar a la derecha
        acall wrLCDcom4
        dec acc
        sjmp crShRt4
ret_scr: ret
; =====
; subrutina placeCur4
; Esta rutina fija la posicion del cursor. La posicion del cursor
; es localizado en el registro b. La posicion del cursor empieza
; en 0. El acumulador da el numero de linea.
; entrada: acumulador indica el numero de linea (1, 2)
; : el registro b guarda la posicion del cursor
; salida : nada
; =====
;
placeCur4:
        dec acc ; acc=0 para linea=1
        jnz line2 ; si acc=0 luego primera linea
        mov a, b
        add a, #080h ; palabra de control para la linea 1
        sjmp setcur
line2:
        mov a, b
        add a, #0C0h ; palabra de control para la linea 2
setcur:
        mov r0, a ; localizar palabra de control
        acall wrLCDcom4 ;
        ret

```

```

; =====
; subrutina dspShLf4
; Esta rutina traslada los contenidos del LCD a la izquierda. El
; numero de caracteres a ser trasladados es localizado en el
; acumulador.
; entrada : acumulador indica el numero de caracteres a trasladar
; salida : nada
; =====
;
dspShLf4:
    jz ret_sdl
    mov r0, #shLfDsp ; palabra de control para trasladar a la izquierda
    acall wrLCDcom4
    dec a
    sjmp dspShLf4
ret_sdl: ret
; =====
; subrutina dspShRt4
; Esta rutina traslada los contenidos del LCD a la derecha. El
; numero de caracteres a ser trasladados es localizado en el
; acumulador.
; entrada : acumulador indica el numero de caracteres a trasladar
; salida : nada
; =====
;
dspShRt4:
    jz ret_sdr
    mov r0, #shRtDsp ; palabra de control para trasladar a la derecha
    acall wrLCDcom4
    dec a
    sjmp dspShRt4
ret_sdr: ret
; =====
; subrutina wrLCDdata4
; escribe un dato al LCD
; el dato debe ser localizado en r0 para llamar el programa
; -----
wrLCDdata4:
    clr LCD_E
    setb LCD_RS ; selecciona enviar Dato
    clr LCD_RW ; selecciona operacion de escritura
    push acc ; salva acumulador
    mov a, r0 ; pone byte de datos en el acc
    mov c, acc.4 ; carga high nibble sobre el bus de datos
    mov LCD_DB4, c ; un bit a la vez...
    mov c, acc.5 ;
    mov LCD_DB5, c
    mov c, acc.6
    mov LCD_DB6, c
    mov c, acc.7
    mov LCD_DB7, c
    setb LCD_E ; pone a uno el pin Enable
    clr LCD_E ; pone a cero el pin Enable
    mov c, acc.0 ; similarmente, carga el low nibble
    mov LCD_DB4, c
    mov c, acc.1
    mov LCD_DB5, c
    mov c, acc.2
    mov LCD_DB6, c
    mov c, acc.3
    mov LCD_DB7, c
    setb LCD_E ; pone a uno el pin Enable
    clr LCD_E ; pone a cero el pin Enable
    lcall retardo
    pop acc
    ret
; =====
; subrutina wrLCDcom4
; escribe una palabra de comando al LCD

```

```

; comando debe estar localizado en r0 para llamar el programa
; -----
wrLCDcom4:
clr LCD_E
clr LCD_RS ; selecciona enviar comando
clr LCD_RW ; selecciona operaci3n de escritura
push acc ; salva el acumulador
mov a, r0 ; pone el byte de datos en el acc
mov c, acc.4 ; carga el high nibble en el bus de datos
mov LCD_DB4, c ; un bit a la vez usando...
mov c, acc.5 ; operaciones de mover bit
mov LCD_DB5, c
mov c, acc.6
mov LCD_DB6, c
mov c, acc.7
mov LCD_DB7, c
setb LCD_E ; pulsa la l3nea Enable
clr LCD_E
mov c, acc.0 ; similarmente, carga el low nibble
mov LCD_DB4, c
mov c, acc.1
mov LCD_DB5, c
mov c, acc.2
mov LCD_DB6, c
mov c, acc.3
mov LCD_DB7, c
setb LCD_E ; pulsa la l3nea Enable
clr LCD_E
lcall retardo
pop acc
ret
; =====
; subrutina initLCD4 - inicializa el LCD
;
; -----
initLCD4:
clr LCD_RS ; LCD linea de Register Select
clr LCD_RW ; linea de Read / Write
clr LCD_E ; linea de Enable
mov r0, #entryMode ; fijar Entry Mode
acall wrLCDcom4 ; incrementar cursor a la derecha, no traslada el display
mov r0, #onDsp ; Enciende el display, home cursor
lcall wrLCDcom4
ret
; =====
; subrutina prtLCD4
; Toma la cadena inmediatamente que sigue a la llamada a esta
; subrutina y lo displaya sobre el LCD. La cadena es leida con la
; instrucc3n mov a, @a+dptr
; de este modo, la cadena esta en la memoria de datos.
;
; entrada : nada
; salida : nada
; destruye : acc, dptr
; =====
;
prtLCD4:
pop dph ; regresa direcci3n de retorno en dptr
pop dpl
prtNext:
clr a ; set offset = 0
movc a, @a+dptr ; consigue chr de la memoria de c3digo
cjne a, #0, chrOK ; si chr = 0 entonces retorna
sjmp retPrtLCD
chrOK:
mov r0, a
acall wrLCDdata4 ; enviar caracter
inc dptr ; apuntar al siguiente caracter
ajmp prtNext ; lazo hasta que finalize la cadena

```

```
retPrtLCD:  
mov a, #1h ; apunta a la instrucci3n despu,s de la cadena  
jmp @a+dptr ; retorno con una instrucci3n de salto  
  
end
```